

**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**

PROIECT IPDP

Studenti:

Ardeleanu Stefan

Cilichidreanu Andi

BRAȘOV, 2025

Departamentul: Electronică și Calculatoare

Programul de studii: Calculatoare

Ardeleanu Stefan
Cilichidreanu Andi

Food Delivery Website

Brașov, 2025

LISTA DE FIGURI, TABELE ȘI CODURI SURSĂ

FIGURI

- Figură 1. App.jsx
- Figură 2. Imports din App.jsx
- Figură 3. Definirea funcției principale App
- Figură 4. Logica de afișare LoginPopup
- Figură 5. Navbar.jsx
- Figură 6. LoginPopup_1.jsx
- Figură 7. LoginPopup_2.jsx
- Figură 8. FoodItem.jsx
- Figură 9. Cart.jsx
- Figură 10. Structura Admin
- Figură 11. Structura Backend
- Figură 12. db.js
- Figură 13. Integrare StripeAPI: crearea unor sesiuni de plată
- Figură 14. Generare token JWT la autentificare
- Figură 15. Middleware-ul protejează rutele de acces neautorizat
- Figură 16. Utilizarea middleware-ului pentru ruta "userorders"
- Figură 17. Pagina principală a aplicației
- Figură 18. Fereastra de autentificare
- Figură 19. Interfața Navbar, după autentificare
- Figură 20. Navigarea prin meniu și afișarea produselor dintr-o categorie selectată
- Figură 21. Adăugarea unui produs în coș și modificarea cantității
- Figură 22. Verificarea coșului de cumpărături
- Figură 23. Interfața Stripe pentru procesarea plății
- Figură 24. Pagina de "MyOrders"

LISTA DE ACRONIME

API – Application Programming Interface;
REST – Representation State Transfer;
HTTP – Hypertext Transfer Protocol;
HTTPS – Hypertext Transfer Protocol Secure;
JSON – JavaScript Object Notation;
AWS – Amazon Web Services;
CI/CD – Continuous Integration/Continuous Delivery;
NoSQL – Not Only SQL;
PCI DSS – Payment Card Industry Data Security Standard;
URL – Uniform Resource Locator;
CSS – Cascading Style Sheets;
DB – Database;
HTML – HyperText Markup Language;
JS – JavaScript;
MVC – Model-View-Controller;
UI/UX – User Interface/User
Experience;

1 INTRODUCERE

1.1 TEMĂ DE PROIECTARE

Cu toții am avut momente în care, obosiți după o zi lungă sau prinși între sarcini, am apelat la o soluție simplă: comanda de mâncare online. Într-o lume în care timpul devine tot mai prețios, platformele de food delivery au devenit parte din rutina noastră zilnică, oferindu-ne rapid și eficient acces la preparatele preferate, direct la ușă.

Mâncarea, dincolo de a fi o nevoie de bază, este o expresie a culturii și a stilului de viață modern. Evoluția tehnologică a transformat modul în care interacționăm cu restaurantele, iar digitalizarea serviciilor de livrare a devenit o necesitate, nu doar un avantaj.

Am ales să dezvoltăm acest proiect deoarece am vrut să îmbinăm utilul cu pasiunea pentru tehnologie și web development. Crearea unui website de food delivery este o oportunitate de a răspunde unei nevoi reale din societate, dar și de a aplica concret cunoștințele acumulate în timpul studiilor.

Totodată, considerăm că acest tip de proiect reflectă foarte bine tendințele actuale, în care digitalul redefineste modul în care trăim, consumăm și ne raportăm la lucrurile esențiale din viața de zi cu zi.

1.2 OBIECTIVELE ȘI SCOPUL LUCRĂRII

Scopul principal al acestei lucrări a fost realizarea unui website de tip food delivery, prin intermediul căruia utilizatorii pot vizualiza un meniu, plasa comenzi și interacționa într-un mod simplu cu platforma. Proiectul are un rol important în consolidarea cunoștințelor dobândite în cadrul studiilor universitare, în special în domeniul dezvoltării web.

Prin acest proiect, am urmărit să aplicăm noțiuni teoretice legate de programare front-end și back-end, lucrul cu baze de date, conceperea unei interfețe prietenoase pentru utilizator, precum și integrarea logicii de funcționare într-un sistem complet funcțional. Am dorit să trecem prin toate etapele dezvoltării unei aplicații web: de la proiectare, structurare și implementare, până la testare și documentare, pentru a înțelege mai bine provocările implicate în realizarea unui produs software.

Am ales tema food delivery deoarece este un domeniu accesibil, cu aplicații directe în viața de zi cu zi. Proiectul reprezintă o oportunitate de a valida într-un mod practic cunoștințele în dezvoltarea web acumulate până la momentul actual și de a experimenta procesul de dezvoltare software într-un cadru clar, definit și realist.

1.3 STRUCTURA PE CAPITOLE

Structura pe capitole a proiectului este următoarea:

- Capitolul 1: Introducere
- Capitolul 2: Fundamente Teoretice
- Capitolul 3: Tehnologii Folosite
- Capitolul 4: Implementarea ?
- Capitolul 5: Implementarea Aplicației Web
- Capitolul 6: Cazuri de Utilizare
- Capitolul 7: Concluzii
- Capitolul 8: Bibliografie

2 FUNDAMENTE TEORETICE

În cadrul acestui capitol sunt prezentate conceptele teoretice și tehnologiile fundamentale care stau la baza dezvoltării unei aplicații web moderne pentru livrarea mâncării. Proiectul se bazează pe arhitectura full-stack JavaScript, folosind React pentru front-end, Node.js și Express pentru back-end, MongoDB pentru stocarea datelor și Stripe pentru procesarea plăților. În plus, sunt abordate concepte precum arhitectura 3-tier, modelul MVC, RESTful APIs și bune practici în dezvoltarea aplicațiilor web.

2.1 INTRODUCERE ÎN DEZVOLTAREA WEB

Dezvoltarea web reprezintă procesul de creare a aplicațiilor și site-urilor accesibile prin intermediul internetului. Aceasta combină mai multe discipline – de la programare și arhitectură software, până la design grafic și experiența utilizatorului. Aplicațiile moderne de tip web sunt din ce în ce mai interactive și complexe, servind scopuri comerciale, educaționale, sociale și industriale. Proiectele actuale presupun nu doar afișarea de informații, ci și interacțiuni avansate precum autentificarea utilizatorilor, procesarea comenzilor, plăți online sau personalizarea conținutului.

2.2 ARHITECTURA CLIENT-SERVER

La baza oricărei aplicații web se află arhitectura **client-server**. Aceasta presupune existența a două componente majore:

- **Clientul:** partea vizibilă utilizatorului (frontend), ce rulează în browser și este responsabilă cu afișarea interfeței grafice, colectarea input-ului și trimiterea cererilor către server.
- **Serverul:** partea invizibilă (backend), care procesează cererile venite de la client, accesează baza de date și returnează răspunsuri.

Comunicarea între client și server se face prin intermediul protocolului **HTTP/HTTPS**, folosind cereri de tip GET, POST, PUT, DELETE etc. Datele sunt de regulă transmise în format **JSON**.

2.3 FRONTEND–UL APLICATIILOR WEB

Frontend–ul unei aplicații web este componenta care definește modul în care utilizatorul interacționează cu aplicația. Această parte este responsabilă pentru prezentare și interfață și este dezvoltată folosind tehnologii precum:

- **HTML (HyperText Markup Language)** – limbajul de bază care definește structura paginilor web;
- **CSS (Cascading Style Sheets)** – stilizează și formatează elementele HTML, controlând aspectul vizual;
- **JavaScript** – limbaj de programare care oferă interactivitate și dinamism paginilor (ex: animații, validări, actualizări de conținut fără reîncărcare).

În ultimii ani, dezvoltarea frontend–ului a evoluat prin apariția **bibliotecilor și framework–urilor moderne**, care permit construirea de aplicații web complexe, bazate pe componente reutilizabile și rutare internă.

2.4 BACKEND–UL APLICATIILOR WEB

Backend–ul reprezintă logica aplicației, rulată pe server, și este responsabil cu:

- procesarea datelor primite de la utilizator,
- gestionarea autentificării și autorizării,
- salvarea și extragerea datelor din baza de date,
- realizarea comunicării cu servicii externe (ex: procesatori de plăți, API-uri).

Limbajele și framework–urile utilizate în dezvoltarea backend–ului variază (ex: PHP, Python, Node.js, Java, Ruby), iar o bună practică este folosirea unui framework web care simplifică definirea rutelor, middleware–urilor și validarea cererilor.

2.5 BAZE DE DATE

Baza de date este componenta în care sunt stocate permanent informațiile relevante ale aplicației: utilizatori, produse, comenzi, mesaje etc. În funcție de tipul de proiect, pot fi folosite baze de date:

- **relaționale** (ex: MySQL, PostgreSQL) – folosesc tabele și relații între entități;
- **NoSQL** (ex: MongoDB, Firebase, CouchDB) – folosesc documente sau colecții, oferind flexibilitate și scalabilitate mai mare pentru aplicații cu structură de date variabilă.

Baza de date este conectată la backend, care o interoghează folosind ORM–uri (Object–Relational Mapping) sau biblioteci dedicate.

2.6 API-URI SI REST

Un API (Application Programming Interface) permite interacțiunea dintre diferite componente software. În contextul aplicațiilor web, cel mai utilizat tip de API este RESTful API, bazat pe metodele HTTP și pe un set de convenții care definesc rutele și acțiunile corespunzătoare.

Un API REST eficient este:

- stateless (nu menține starea între cereri),
- scalabil (permite interacțiuni de la multiple surse),
- structurat (urmărește convenții standardizate pentru numele rutelor și tipul răspunsurilor).

Prin intermediul API-urilor REST, frontend-ul trimite cereri către backend pentru a obține sau modifica date, fără a fi nevoie de o conexiune directă la baza de date.

2.7 AUTENTIFICARE SI SECURITATE

Aplicațiile moderne, mai ales cele care implică date personale sau tranzacții financiare, necesită mecanisme solide de autentificare și autorizare. Acestea sunt realizate prin:

- Formular de login și parole criptate (ex: bcrypt);
- Token-uri de autentificare (ex: JWT – JSON Web Tokens);
- Protecție împotriva atacurilor comune: Cross-Site Scripting (XSS), SQL Injection, Cross-Site Request Forgery (CSRF).

Securitatea este o componentă critică a aplicațiilor web și trebuie avută în vedere încă din faza de proiectare.

2.8 UX SI UI IN DEZVOLTAREA WEB

UX (User Experience) și UI (User Interface) sunt concepte esențiale pentru succesul unei aplicații web. O interfață plăcută vizual și ușor de utilizat contribuie semnificativ la satisfacția utilizatorului.

- UI se referă la aspectul vizual (culori, fonturi, butoane, layout).
- UX are în vedere traseul utilizatorului, simplitatea acțiunilor, viteza și claritatea interacțiunii.

În cazul unei aplicații de food delivery, experiența utilizatorului trebuie să fie intuitivă, rapidă și fără pași inutili – de la alegerea produsului până la finalizarea comenzii.

2.9 HOSTING SI DEPLOYMENT

După dezvoltare, aplicația trebuie publicată online printr-un proces numit deployment. Hosting-ul poate fi de mai multe tipuri:

- static (ex: Vercel, Netlify) – pentru aplicații frontend;

- dinamic/cloud (ex: Heroku, Render, AWS) – pentru backend și baze de date.

Un proces eficient de deployment include:

- versiuni de producție și testare (dev/prod),
- sistem CI/CD pentru actualizări automate,
- monitorizarea erorilor și logurilor.

3.1 HTML

HTML (HyperText Markup Language) reprezintă fundamentul dezvoltării paginilor web. Este un limbaj de marcare utilizat pentru a structura conținutul într-un document accesibil în browser. Fiecare pagină web vizibilă pe internet conține elemente HTML, care definesc diferitele componente ale interfeței – de la paragrafe, titluri și liste, până la imagini, formulare și butoane interactive.

HTML nu este un limbaj de programare, ci un limbaj de structurare, fiind responsabil pentru organizarea logică a informației. Utilizarea tagurilor (etichete) permite dezvoltatorilor să creeze o ierarhie clară a conținutului. De exemplu, tagul `<h1>` indică un titlu de nivel înalt, în timp ce `<p>` marchează un paragraf.

Pe parcursul evoluției sale, HTML a suferit numeroase modificări, cea mai recentă versiune stabilă fiind HTML5. Aceasta a introdus elemente semantice precum `<header>`, `<footer>`, `<section>` și `<article>`, care contribuie la o mai bună înțelegere a structurii de către browsere și motoarele de căutare. HTML5 aduce de asemenea suport pentru multimedia (video și audio native), API-uri moderne (ex. Geolocation, Drag and Drop) și o mai bună integrare cu JavaScript.

Din punct de vedere al rolului său în dezvoltarea aplicației actuale, HTML stă la baza interfeței cu utilizatorul, permițând structurarea meniurilor, paginilor cu produse, coșului de cumpărături sau formularelor de autentificare.

3.2 JAVASCRIPT

JavaScript este limbajul de programare principal utilizat în dezvoltarea web pentru a adăuga interactivitate și logică aplicațiilor web. Spre deosebire de HTML și CSS, care sunt limbaje de marcare și stilizare, JavaScript este un limbaj de scripting orientat pe obiecte, ce permite manipularea DOM (Document Object Model), comunicarea cu serverele, validarea formularelor, animarea elementelor și multe altele.

Un aspect important este faptul că JavaScript este un limbaj de interpretare, rulând direct în browser. Totuși, cu apariția tehnologiilor moderne precum Node.js, acesta poate fi folosit și pe partea de backend, oferind astfel posibilitatea construirii de aplicații full-stack doar cu JavaScript.

Printre caracteristicile esențiale ale JavaScript se numără:

- Suport pentru programare asincronă prin `async/await` și `Promises`;
- Manipulare DOM pentru actualizarea dinamică a conținutului;
- Comunicarea cu API-uri externe prin `fetch()` sau `axios`;

- Posibilitatea modularizării codului și utilizarea framework-urilor precum React, Angular, Vue etc.

În proiectul de față, JavaScript este limbajul central folosit pentru dezvoltarea logicii aplicației. În combinație cu React, JavaScript oferă o bază solidă pentru construirea unei aplicații rapide, dinamice și scalabile, în care fiecare interacțiune a utilizatorului este gestionată eficient.

3.3 CSS

CSS (Cascading Style Sheets) este tehnologia utilizată pentru a controla aspectul vizual al paginilor HTML. Dacă HTML se ocupă de „ce este afișat”, CSS răspunde la întrebarea „cum este afișat”. Prin CSS, dezvoltatorii pot aplica stiluri precum culori, dimensiuni, poziționare, fonturi și animații asupra elementelor HTML.

CSS funcționează prin aplicarea de reguli stilistice asupra unor selecții de elemente HTML. Aceste reguli pot fi aplicate în mod intern (în cadrul unui fișier HTML), inline (direct în elementul HTML), dar cel mai recomandat este utilizarea fișierelor CSS externe, pentru a separa clar partea de prezentare de cea de structură.

CSS a evoluat considerabil, iar în prezent standardul CSS3 oferă dezvoltatorilor o gamă largă de funcționalități moderne:

- Flexbox și Grid pentru layout-uri responsive;
- Media queries pentru design adaptabil la diferite dispozitive;
- Animații și tranziții CSS pentru o interacțiune mai fluentă;
- Pseudo-clase (:hover, :nth-child, etc.) pentru stiluri dinamice fără JavaScript.

În cadrul proiectului curent, CSS este esențial pentru obținerea unui design intuitiv, coerent și responsiv. Interfața aplicației de food delivery trebuie să fie atrăgătoare atât pe desktop, cât și pe mobil.

Astfel, utilizarea intensă a CSS ajută la construirea unui user experience plăcut și profesionist.

3.4 REACT

React este o bibliotecă JavaScript open-source, dezvoltată de Facebook, folosită pentru construirea interfețelor de utilizator interactive și reactive. React este bazat pe conceptul de componente reutilizabile care pot fi combinate pentru a construi aplicații complexe.

Unul dintre cele mai importante concepte ale React este DOM-ul virtual (Virtual DOM) – o reprezentare în memorie a structurii paginii, care permite React să actualizeze doar porțiunile modificate ale interfeței în loc să reîncarce întreaga pagină. Acest lucru duce la o performanță mai bună și o experiență fluidă pentru utilizator.

Alte concepte cheie ale React includ:

- JSX (JavaScript XML) – o sintaxă ce permite scrierea codului HTML direct în JavaScript;
- State și Props – pentru gestionarea datelor dinamice și comunicarea între componente;
- Hooks (useState, useEffect, etc.) – pentru gestionarea logicii componente într-un mod curat și declarativ;
- Router – pentru navigare între pagini (ex: React Router).

În cadrul proiectului, React este utilizat pentru realizarea interfeței aplicației – pagina principală cu produsele, coșul de cumpărături, pagina de finalizare comandă, autentificare, etc. Folosirea React asigură o interfață modernă, modulară și ușor de întreținut.

3.5 NODE.JS

Node.js este un mediu de execuție JavaScript construit pe motorul V8 al browserului Chrome, care permite rularea codului JavaScript în afara browserului, adică pe partea de server (backend). Această caracteristică revoluționară a permis dezvoltatorilor web să folosească JavaScript pentru întregul stack – atât client, cât și server –, reducând nevoia de a învăța alte limbaje precum PHP, Java sau Python pentru dezvoltarea backend-ului.

Node.js funcționează pe un model de execuție non-blocantă, asincronă, ceea ce înseamnă că poate gestiona un număr foarte mare de cereri simultane fără a bloca firul principal de execuție. Acest model de I/O (input/output) bazat pe evenimente este ideal pentru aplicații web care trebuie să fie scalabile și rapide, precum aplicații de livrare mâncare, unde utilizatorii pot interacționa simultan cu sistemul (comenzi, plăți, actualizări de status etc.).

Alte caracteristici importante ale Node.js includ:

- npm (Node Package Manager) – un ecosistem uriaș de pachete reutilizabile;
- Cross-platform – rulează pe Windows, Linux, Mac;
- Performanță ridicată datorită motorului V8;
- Ideal pentru REST APIs, aplicații în timp real (chat, livrări), microservicii.

În cadrul proiectului actual, Node.js servește drept infrastructură backend, gestionând logica serverului, conexiunile cu baza de date și procesarea comenzilor.

3.6 EXPRESS.JS

Express.js este un framework minimalist și flexibil pentru Node.js care simplifică procesul de creare a serverelor web și API-urilor. Deși Node.js permite crearea unui server web de la zero, Express oferă o serie de facilități predefinite, cum ar fi rutare, middlewares, parsarea request-urilor și multe altele, ceea ce accelerează dezvoltarea și reduce codul boilerplate.

Express permite definirea ușoară a rutelor (endpoint-uri) HTTP (GET, POST, PUT, DELETE), care răspund la cererile clientului și trimit date înapoi, de regulă în format JSON. Totodată, framework-ul oferă suport pentru middleware – funcții care rulează între primirea cererii și trimiterea răspunsului –, utile pentru autentificare, validări sau logare.

Exemple de concepte cheie în Express:

- `app.get('/ruta', funcție)` – pentru gestionarea cererilor GET;

- req și res – obiectele de request și response;
- next() – pentru trecerea la următorul middleware;
- Utilizarea de librării externe precum cors, body-parser, jsonwebtoken.

În proiectul de food delivery, Express este esențial pentru definirea rutei de plasare comandă, autentificare utilizatori, interacțiune cu baza de date și integrarea cu serviciul de plăți.

3.7 MONGODB

MongoDB este un sistem de gestionare a bazelor de date NoSQL, orientat pe documente, care stochează datele în format JSON (mai precis, BSON – Binary JSON). Este o alegere foarte populară pentru aplicații moderne datorită flexibilității și scalabilității sale.

Spre deosebire de bazele de date relaționale (SQL), care stochează datele în tabele rigide, MongoDB permite stocarea de obiecte complexe, în mod nesting, fără a impune o schemă fixă. Acest lucru înseamnă că structura fiecărui document poate varia, ceea ce este util într-o aplicație în continuă schimbare.

Avantajele MongoDB includ:

- Performanță ridicată la scriere și citire;
- Scalabilitate orizontală prin sharding;
- Flexibilitate în structura datelor;
- Suport pentru replicare și toleranță la erori.

În aplicația de food delivery, MongoDB este folosit pentru stocarea utilizatorilor, comenzilor, produselor și istoricului tranzacțiilor. Fiecare document poate reprezenta o comandă cu detalii precum produse, cantitate, preț total, stare (în curs, livrat etc.) și timestamp.

3.8 MONGOOSE

Mongoose este o bibliotecă JavaScript pentru Node.js care oferă un Object Data Modeling (ODM) elegant pentru MongoDB.

Ea creează o punte între codul JavaScript și baza de date MongoDB, permițând dezvoltatorilor să definească modele (schemă) clare pentru documentele din colecții, chiar dacă MongoDB, prin natura sa NoSQL, nu impune o schemă fixă.

Funcționalități importante ale Mongoose:

- Definirea de Scheme (ex: pentru utilizatori, produse, comenzi);
- Validări implicite (ex: câmp obligatoriu, tip de date, unicitate);
- Middleware-uri (hooks) pentru acțiuni pre/post salvare;
- Popularea documentelor asociate (ex: comenzi + produse incluse);
- Operatori MongoDB integrați: \$push, \$pull, \$set, etc.

Mongoose face codul mai clar, mai ușor de întreținut și previne multe erori legate de structurarea datelor.

3.9 STRIPE

Stripe este o platformă de procesare a plăților online, utilizată pe scară largă pentru integrarea rapidă a tranzacțiilor financiare în aplicații web sau mobile. Stripe oferă o interfață prietenoasă pentru dezvoltatori și un set complet de API-uri pentru a gestiona plăți cu cardul, subscripții, rambursări, notificări și altele.

Printre caracteristicile principale ale Stripe se numără:

- Securitate ridicată – Stripe este certificat PCI DSS Level 1;
- Tokenizare – cardurile nu sunt procesate direct, ci prin tokenuri securizate;
- Webhook-uri – pentru notificări în timp real asupra schimbărilor de stare a unei plăți;
- Interfață user-friendly pentru formularele de plată integrate;
- Suport pentru numeroase tipuri de valute și metode de plată.

În aplicația de food delivery, Stripe permite utilizatorului să efectueze plata online în siguranță. După ce utilizatorul finalizează comanda, datele de plată sunt trimise către Stripe, care procesează tranzacția și notifică aplicația dacă aceasta a fost acceptată sau respinsă.

3.10 REACT ROUTER

React Router este o bibliotecă esențială pentru aplicațiile React care necesită navigare între mai multe pagini fără reîncărcare completă. Ea permite gestionarea routing-ului pe partea de client, ceea ce oferă o experiență mult mai fluidă utilizatorilor.

Caracteristici importante:

- Definirea rutelor în React (`<Route path="/checkout" element={<CheckoutPage />} />`);
- Navigare programatică (`useNavigate()` hook);
- Suport pentru parametri dinamici (`/product/:id`);
- Navigație condiționată (ex: dacă user-ul e logat).

React Router folosește un sistem bazat pe URL-uri și componente, astfel încât schimbarea rutei actualizează componenta afișată fără a reîncărca întreaga pagină.

Este utilizat în proiect pentru a permite navigarea între pagina de autentificare, lista produselor, coșul de cumpărături și pagina de plată.

3.11 AXIOS

Axios este o bibliotecă JavaScript foarte populară folosită pentru efectuarea de cereri HTTP de pe partea de client sau server. Este bazată pe Promises și este preferată de mulți dezvoltatori pentru simplitate, suport pentru interceptoare, și manipularea facilă a răspunsurilor și erorilor.

Funcționalități:

- Cereri GET, POST, PUT, DELETE;
- Trimiterea de headers, params, body cu ușurință;
- Interceptarea cererilor și răspunsurilor;
- Integrare ușoară cu async/await.

Axios este utilizat în partea de frontend (React) pentru a comunica cu backend-ul Express: trimiterea comenzilor, autentificarea utilizatorilor, verificarea statusului livrării etc.

3.12 JWT(JSON WEB TOKENS)

JWT (JSON Web Token) este un standard des utilizat pentru transmiterea securizată a informațiilor între două părți – de obicei client și server – sub forma unui obiect JSON codificat. În aplicațiile moderne, JWT este folosit în mod frecvent pentru autentificare, în special în arhitecturile bazate pe REST, unde statelessness este o cerință importantă.

Un token JWT este format din trei părți:

1. Header – specifică algoritmul de semnare și tipul tokenului;
2. Payload – conține informațiile (de exemplu, ID-ul utilizatorului, rolul, email-ul etc.);
3. Signature – semnătura digitală generată pe baza primelor două părți și a unui secret cunoscut doar de server.

JWT este transmis către client după autentificare și este ulterior atașat în header-ul fiecărei cereri HTTP (de regulă în Authorization: Bearer <token>), pentru ca serverul să poată verifica identitatea utilizatorului fără a păstra sesiuni în server.

Avantaje:

- Este stateless – serverul nu trebuie să mențină sesiuni în memorie;
- Token-ul poate conține informații utile despre utilizator;

- Ușor de utilizat în aplicații SPA (Single Page Applications).

Aici, JWT este utilizat pentru a securiza endpoint-urile backend: doar utilizatorii autentificați pot plasa comenzi sau accesa istoricul comenzilor.

3.13 MIDDLEWARE

Middleware-urile sunt funcții intermediare în Express.js care rulează între momentul în care o cerere este primită de server și momentul în care răspunsul este trimis înapoi.

Pot fi folosite pentru:

- Verificare autentificare (ex: `verifyToken`);
- Parsarea corpului cererilor (ex: `express.json()`);
- Logarea activităților;
- Tratarea erorilor.

Middleware-urile contribuie la structurarea logicii aplicației într-un mod modular și reutilizabil, fiind un element important în orice aplicație Express complexă.

3.14 VITEST

Vitest este un framework de testare rapid și modern pentru aplicații scrise în JavaScript/TypeScript, creat de echipa Vite. Este similar cu Jest, dar se integrează foarte bine cu ecosistemul Vite. Am ales Vitest pentru că oferă timpi de rulare foarte mici la testare, suportă mocking, snapshot testing și funcționează perfect în proiecte construite cu Vite (care a fost folosit și în proiectul meu). Am scris teste unitare pentru componentele principale și funcțiile critice din aplicație. De asemenea, am folosit Vitest împreună cu Chai pentru aserțiuni.

Beneficii:

- Integrare nativă cu Vite
- Timp rapid de execuție a testelor
- Compatibilitate cu TypeScript
- Suport pentru mocking și snapshot testing

3.15 JEST

Jest este un framework de testare dezvoltat de Facebook, foarte popular în ecosistemul React, dar folosit și pentru alte aplicații JavaScript. Oferă testare unitară, de integrare și mocking. L-am folosit pentru a compara performanța cu Vitest sau pentru a testa părți din aplicație care nu erau compatibile cu Vitest.

3.16 GITHUB ACTIONS

GitHub Actions este un serviciu de automatizare CI/CD (Continuous Integration / Continuous Deployment) oferit de GitHub. Permite rularea de scripturi automate la anumite evenimente (push, pull request, etc.). Am configurat un workflow pentru a rula testele automat la fiecare push în branch-ul principal. Astfel, am asigurat că aplicația nu este afectată de bug-uri introduse prin commit-uri noi.

Beneficii:

- Automatizarea testării și build-ului
- Prevenirea integrării de cod care strică funcționalitatea
- Feedback rapid pentru dezvoltatori

3.17 CHAI

Chai este o librărie de aserțiuni pentru JavaScript, frecvent folosită împreună cu Mocha, Vitest sau alte frameworkuri de testare. Oferă o sintaxă expresivă, în stil BDD/TDD. Am folosit Chai pentru că oferă o sintaxă clară pentru aserțiuni, cum ar fi `expect(value).to.equal(...)`, ceea ce face testele mai ușor de citit și întreținut. Chai a fost integrat în testele scrise cu Vitest. Exemple de teste includ validarea funcțiilor de procesare a datelor, comportamentul componentelor etc.

Beneficii:

- Sintaxă intuitivă și expresivă
- Integrare ușoară cu Vitest
- Suportă diverse stiluri de aserțiuni: `expect`, `should`, `assert`

4 IMPLEMENTAREA SITE-ULUI WEB

4.1 ARHITECTURA

4.1.1 Prezentare generala

Aplicația web de tip food delivery este construită pe baza unei arhitecturi moderne de tip **client-server**, separând clar responsabilitățile între partea de interfață cu utilizatorul (**frontend**) și logica aplicației și gestionarea datelor (**backend**).

Această abordare facilitează o mai bună scalabilitate, întreținere și organizare a codului, permițând dezvoltarea și testarea independentă a fiecărei componente majore.

La nivel înalt, arhitectura aplicației se poate împărți în următoarele componente majore:

- **Frontend (Clientul web)**

Este partea vizibilă pentru utilizator, implementată folosind **React.js**, oferă o experiență dinamică și interactivă, fără a fi necesară reîncărcarea paginii. Interacțiunile utilizatorului (ex: adăugarea produselor în coș, plasarea comenzilor) sunt transmise către backend prin apeluri HTTP.

- **Backend (Serverul web)**

E partea care gestionează logica de business, operațiunile cu baza de date și procesarea plăților. Este implementat folosind **Node.js** și **Express.js**, două tehnologii populare în ecosistemul JavaScript care permit dezvoltarea rapidă a aplicațiilor RESTful. Serverul răspunde cererilor frontend-ului prin expunerea unor endpoint-uri HTTP (API-uri), autentifică utilizatorii, gestionează comenzile și comunică cu baza de date.

- **Baza de date**

Aplicația folosește **MongoDB**, o bază de date NoSQL orientată pe documente, potrivită pentru stocarea flexibilă a datelor despre produse, utilizatori și comenzi. Datorită naturii sale schemaless, MongoDB permite o adaptare ușoară la eventuale schimbări în structura datelor, ceea ce este avantajos în etapa de dezvoltare.

- **Serviciul de procesare plăți – Stripe** – pentru a gestiona plățile online într-un mod securizat și ușor de implementat, aplicația este integrată cu **Stripe API**. Acesta permite utilizatorilor să efectueze plăți prin card bancar, oferind un proces de checkout fluid și sigur. Stripe este apelat din backend, dar inițiat din frontend, unde utilizatorul introduce datele de plată.

Fluxul general de date în cadrul aplicației este următorul:

1. Utilizatorul accesează aplicația prin browser, interacționând cu interfața React.
2. La plasarea unei comenzi sau autentificare, frontend-ul trimite cereri către backend prin API-uri REST.
3. Backend-ul prelucrează cererile, verifică și salvează informațiile în baza de date MongoDB sau interacționează cu Stripe pentru procesarea plății.
4. Răspunsul este trimis înapoi către frontend, care actualizează interfața în funcție de rezultat (confirmare comandă, afișare produse, autentificare etc.).

Aplicația definește două tipuri principale de utilizatori:

- **Clientul** – utilizatorul final care navighează site-ul, adaugă produse în coș, se autentifică și plasează comenzi.
- **Administratorul** – are acces la un panou dedicat unde poate gestiona produsele din meniu (adăugare, modificare, ștergere) și vizualiza comenzile primite.

Această separare între componentele aplicației și între tipurile de utilizatori contribuie la o mai bună organizare logică a funcționalităților și la o experiență de utilizare clară și intuitivă. În plus, arhitectura modulară permite extinderea ușoară a proiectului în viitor (ex: integrarea de notificări, tracking în timp real sau aplicație mobilă).

4.2 FRONTEND

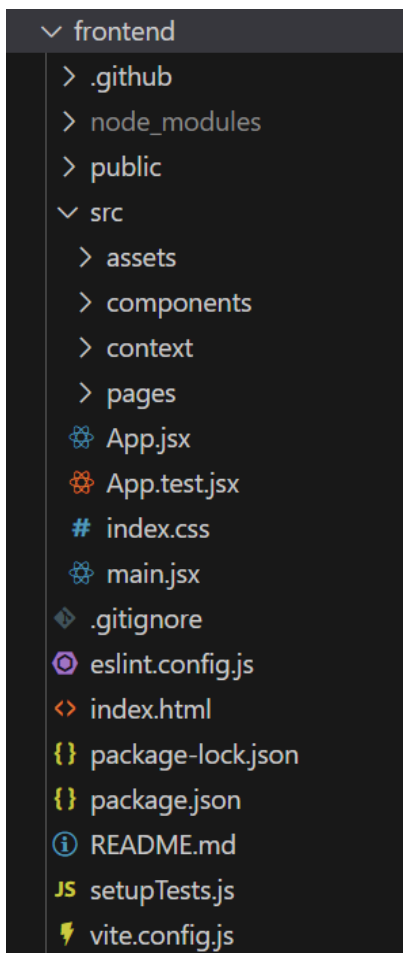
Frontend-ul reprezintă partea vizibilă a aplicației de food delivery, adică interfața cu care utilizatorii interacționează direct.

Pentru realizarea acestei componente, s-a utilizat biblioteca **React**, una dintre cele mai populare tehnologii pentru dezvoltarea de interfețe moderne și dinamice. Prin intermediul React, aplicația oferă o experiență fluentă, reactivă și modulară.

4.2.1 Structura generală a Frontend-ului

Frontend-ul este organizat pe baza unei structuri modulare, în care fiecare componentă are propriul fișier/folder, facilitând mentenanța și extinderea aplicației.

Structura folderelor este intuitivă și separă clar logica aplicației de partea de stilizare sau de date:



src/

Acesta este folderul principal în care se află codul sursă al aplicației. Este împărțit pe module logice:

- assets

Conține fișiere statice precum imagini, logo-uri, iconițe, sau alte resurse media utilizate în interfață.

- components

include componentele React reutilizabile (ex. Navbar, Footer, ProductCard etc.). Acestea sunt unități de interfață mici și izolate, fiecare având propriul fișier .jsx și uneori fișier de stil separat (.css sau .scss).

- context

Aici sunt definite fișierele care folosesc React Context API pentru gestionarea stării globale a aplicației (ex: coșul de cumpărături, utilizatorul autentificat). Se folosesc împreună cu hook-uri ca useReducer și useContext.

- pages

Conține paginile mari ale aplicației, fiecare implementând o anumită secțiune funcțională:

Home.jsx, Login.jsx, Cart.jsx, AdminDashboard.jsx etc.

App.jsx

Acesta este fișierul principal care orchestrează toate componentele și definește rutele aplicației folosind React Router. Este punctul de plecare pentru logica de routing și layout global (ex: afișarea Navbar-ului pe toate paginile).

App.test.jsx / setupTests.js

Fișiere asociate testării aplicației cu instrumente precum Jest sau React Testing Library. Deși nu sunt utilizate în toate proiectele, acestea sunt utile pentru validarea automată a componentelor.

main.jsx

Este fișierul de „bootstrap” al aplicației. Aici este randată aplicația React în DOM-ul HTML, folosind funcția ReactDOM.createRoot() și componenta <App />

index.css

Fișier global de stilizare. Aici pot fi definite stiluri de bază, reguli pentru fonturi, culori, spațieri etc. Se aplică întregii aplicații.

index.html

Se află în folderul public și reprezintă șablonul HTML de bază în care React va încărca aplicația. Este un fișier esențial care conține un singur div cu id="root" – locul unde React montează aplicația.

package.json & vite.config.js

- package.json gestionează toate pachetele și dependențele frontend-ului, inclusiv scripturile de build și start.
- vite.config.js este fișierul de configurare al framework-ului Vite, folosit pentru dezvoltarea rapidă și build performant.

4.2.2 Componenta principală App.jsx

Fișierul App.jsx reprezintă componenta principală a aplicației de frontend, responsabilă pentru structurarea paginilor și gestionarea afișării componentelor esențiale.

Este punctul de pornire în aplicația React și conține logica de routing și elementele globale precum bara de navigație (Navbar) și footer-ul (Footer).

Mai jos este prezentată o captură de ecran a codului complet din fișierul App.jsx:

```
1  import React from 'react'
2  import Navbar from './components/Navbar/Navbar'
3  import { Route, Routes } from 'react-router-dom'
4  import Home from './pages/Home/Home'
5  import Cart from './pages/Cart/Cart'
6  import PlaceOrder from './pages/PlaceOrder/PlaceOrder'
7  import Footer from './components/Footer/Footer'
8  import LoginPopup from './components/LoginPopup/LoginPopup'
9  import { useState } from 'react'
10
11  const App = () => {
12
13    const [showLogin, setShowLogin] = useState(false)
14
15    return (
16      <>
17        {showLogin?<LoginPopup setShowLogin={setShowLogin}/>:<></> }
18        <div className='app'>
19          <Navbar setShowLogin={setShowLogin}/>
20          <Routes>
21            <Route path='/' element={<Home/>} />
22            <Route path='/cart' element={<Cart/>} />
23            <Route path='/order' element={<PlaceOrder/>} />
24          </Routes>
25        </div>
26        <Footer/>
27      </>
28    )
29  }
30
31  export default App
32
33
```

Figura 1 App.jsx

Se importă toate componentele necesare pentru interfața principală: paginile (Home, Cart, PlaceOrder), componentele comune (Navbar, Footer) și popup-ul de login (LoginPopup).

Se folosește useState din React pentru gestionarea stării aplicației.

Routes și Route sunt importate din react-router-dom pentru a defini rutele aplicației în stil SPA (Single Page Application).

```

1  import React from 'react'
2  import Navbar from './components/Navbar/Navbar'
3  import { Route, Routes } from 'react-router-dom'
4  import Home from './pages/Home/Home'
5  import Cart from './pages/Cart/Cart'
6  import PlaceOrder from './pages/PlaceOrder/PlaceOrder'
7  import Footer from './components/Footer/Footer'
8  import LoginPopup from './components/LoginPopup/LoginPopup'
9  import { useState } from 'react'

```

Figura 2 Imports din App.jsx

Se definește componenta funcțională App.

Se declară un state numit showLogin, care controlează dacă fereastra de login este afișată sau nu. Valoarea inițială este false.

```

11  const App = () => {
12
13    const [showLogin, setShowLogin] = useState(false)

```

Figura 3 Definirea funcției principale App

Dacă showLogin este true, se afișează componenta LoginPopup.

Se trimite ca prop funcția setShowLogin pentru ca popup-ul să o poată închide.

```

{showLogin?<LoginPopup setShowLogin={setShowLogin}/>:<></>}

```

Componenta Navbar este afișată în permanență, iar setShowLogin este transmis pentru a putea activa popup-ul din bara de navigație.

Structura Routes definește toate paginile aplicației:

- Pagina principală (Home)
 - cart – Pagina coșului de cumpărături (Cart)
 - order – Pagina de plasare a comenzii (PlaceOrder)

Navigarea între pagini se face fără reîncărcarea întregii aplicații, datorită arhitecturii SPA oferite de React.

```
18 <div className='app'>
19   <Navbar setShowLogin={setShowLogin}/>
20   <Routes>
21     <Route path='/' element={<Home/>} />
22     <Route path='/cart' element={<Cart/>} />
23     <Route path='/order' element={<PlaceOrder/>} />
24   </Routes>
25 </div>
```

Componenta Footer este afișată la finalul fiecărei pagini și conține informații generale

Exportul implicit al componentei App, pentru a putea fi utilizată în alte părți ale aplicației, în special în fișierul index.js (punctul de intrare al aplicației React).

4.2.3 Componenta Navbar.jsx

Componenta Navbar are rolul de a afișa bara de navigare superioară a aplicației, permițând utilizatorului să acceseze rapid diferite secțiuni, precum pagina principală, coșul de cumpărături, secțiunea de produse și pagina de autentificare.

Este o componentă reutilizabilă, prezentă în majoritatea paginilor aplicației.

Codul partial de mai jos arată structura de navigație:

```

20   return (
21     <div className="navbar">
22       <Link to="/">
23         <img src={assets.logo} alt="company-logo" className="logo" />
24       </Link>
25
26       <ul className="navbar-menu">
27         <Link
28           to="/"
29           onClick={() => setMenu("home")}
30           className={menu === "home" ? "active" : ""}
31           data-testid="home"
32         >
33           home
34         </Link>
35         <a
36           href="#explore-menu"
37           onClick={() => setMenu("menu")}
38           className={menu === "menu" ? "active" : ""}
39           data-testid="menu"
40         >
41           menu
42         </a>
43         <a
44           href="#app-download"
45           onClick={() => setMenu("mobile-app")}
46           className={menu === "mobile-app" ? "active" : ""}
47           data-testid="mobile-app"
48         >
49           mobile-app
50         </a>
51         <a
52           href="#footer"
53           onClick={() => setMenu("contact-us")}
54           className={menu === "contact-us" ? "active" : ""}
55           data-testid="contact-us"
56         >
57           contact us
58         </a>
59       </ul>
60
61       <div className="navbar-right">
62         <img src={assets.search_icon} alt="search-icon" />
63
64         <div className="navbar-search-icon">
65           <Link to="/cart">

```

Figura 6 Navbar.jsx

4.2.4 Componenta LoginPopup.jsx

Componenta LoginPopup este o fereastră modală ce permite utilizatorului să se autentifice în aplicație. Aceasta este afișată dinamic atunci când utilizatorul apasă pe butonul „Login” din bara de navigare (Navbar).

În acest fragment de cod se gestionează starea curentă a formularului (Login sau Sign Up) și se actualizează datele introduse de utilizator în timp real, prin funcția onChangeHandler, folosind hook-ul useState.

```
12  const [currState, setCurrState] = useState("Login")
13  const [data, setData] = useState({
14    name: "",
15    email: "",
16    password: ""
17  });
18
19  const onChangeHandler = (event) => {
20    const name = event.target.name;
21    const value = event.target.value;
22    setData(data => ({...data, [name]: value}));
23  }
```

Figura 7 LoginPopup_1.jsx

Mai departe, funcția onLogin construiește dinamic URL-ul cererii în funcție de starea curentă (autentificare sau înregistrare) și trimite datele utilizatorului către backend printr-un request POST cu axios.

Răspunsul serverului este tratat corespunzător pentru a salva token-ul JWT și a ascunde popup-ul în caz de succes.

Putem observa acestea în porțiunea de cod din figura atasată mai jos:

```

25   const onLogin = async (event) => {
26       event.preventDefault();
27       let newUrl = url;
28       if(currState==="Login"){
29           newUrl += "/api/user/login"
30       }
31       else {
32           newUrl+= "/api/user/register"
33       }
34
35       const response = await axios.post(newUrl, data);
36
37       if(response.data.success){
38           setToken(response.data.token);
39           localStorage.setItem("token", response.data.token);
40           setShowLogin(false);
41       }
42       else {
43           alert(response.data.message);
44       }
45   }
46

```

Figura 8 LoginPopup_2.jsx

Această componentă contribuie semnificativ la experiența utilizatorului, asigurând o metodă simplă și intuitivă de autentificare în aplicație, fără a-l redirecționa către o pagină separată.

4.2.5 Componenta de afisare a produselor

- FoodDisplay.jsx
- FoodItem.jsx

FoodDisplay reprezintă containerul principal pentru afișarea produselor disponibile în platforma de food delivery. Primește o categorie (category) ca prop și afișează doar produsele care corespund acesteia (sau toate, dacă este selectată categoria „All”).

Se folosește contextul global (StoreContext) pentru a accesa lista de produse (food_list) și a o parcurge dinamic folosind map(). Fiecare produs este afișat prin componenta FoodItem.

FoodItem este responsabil pentru afișarea fiecărui produs individual. Conține:

- Imaginea produsului
- Numele, descrierea și prețul

- Buton de adăugare în coș sau control de cantitate (dacă deja există în coș)

Folosește contextul global (StoreContext) pentru a accesa:

- cartItems: obiect ce conține produsele din coș și cantitatea fiecăruia
- Funcțiile addToCart și removeFromCart pentru actualizarea stării

```
{!cartItems[id] ? (  
  <img  
    className="add"  
    onClick={() => addToCart(id)}  
    src={assets.add_icon_white}  
    alt="add-icon"  
  />  
): (  
  <div className="food-item-counter">  
    <img onClick={() => removeFromCart(id)} src={assets.remove_icon_red} alt="remove-item"/>  
    <p>{cartItems[id]}</p>  
    <img onClick={() => addToCart(id)} src={assets.add_icon_green} alt="add-item" />  
  </div>  
)}]
```

Figura 9 FoodItem.jsx

4.2.6 Componenta Cart.jsx

Componenta Cart.jsx este responsabilă pentru afișarea coșului de cumpărături al utilizatorului. Aceasta accesează starea globală definită în StoreContext pentru a obține lista de produse, cantitățile adăugate și funcționalitățile asociate (ex: eliminarea unui produs).

Logica de mapare verifică pentru fiecare produs dacă există în coș (`cartItems[item._id] > 0`) și, dacă da, îl afișează împreună cu imaginea, denumirea, prețul unitar, cantitatea, totalul individual și opțiunea de eliminare.

În partea de jos a componentei, se face un rezumat al coșului:

- Subtotal calculat prin funcția `getTotalCartAmount()`
- Taxa de livrare condiționată de valoarea coșului
- Totalul final afișat dinamic

De asemenea, este oferită posibilitatea introducerii unui cod promoțional și plasarea comenzii prin redirectarea către ruta `/order`.

Cod relevant:

```

{food_list.map((item, index) => {
  if (cartItems[item._id] > 0) {
    return (
      <div data-testid={`cart-item-${item._id}`} >
        <div className="cart-items-title cart-items-item" >
          <img src={`url+"/images/"+item.image`} alt={item.name} />
          <p>{item.name}</p>
          <p data-testid={`cart-item-price-${item._id}`}>{item.price}</p>
          <p data-testid={`cart-item-qty-${item._id}`}>{cartItems[item._id]}</p>
          <p data-testid={`cart-item-total-${item._id}`}>{item.price * cartItems[item._id]}</p>
          <p data-testid = {`cart-item-remove-${item._id}`} onClick={() => removeFromCart(item._id)} className="cross" >x</p>
        </div>
      <hr />
    </div>
  );
})]

```

Figura 10 Cart.jsx

4.2.7 Alte componente

Pentru completarea funcționalității aplicației de food delivery, am utilizat mai multe componente prezentate pe scurt mai jos:

- **Header.jsx**
Această componentă are rol de secțiune introductivă, fiind poziționată în partea superioară a paginii principale.
Afișează un mesaj de tip hero banner (ex. „Livrăm mâncarea preferată la tine acasă”) și include un câmp de căutare pentru produse. Nu conține logică funcțională complexă, ci doar markup HTML și stilizare.
- **ExploreMenu.jsx**
ExploreMenu oferă o interfață vizuală pentru filtrarea produselor în funcție de categorie (de exemplu: „Pizza”, „Burgeri”, „Deserturi”). Este construită ca o listă de butoane sau carduri, care setează categoria selectată. Această selecție influențează conținutul afișat în componenta FoodDisplay.
- **Footer.jsx**
Footer-ul este o componentă pur vizuală, poziționată la finalul paginii. Afișează link-uri utile (ex. „Termeni și condiții”, „Politica de confidențialitate”) și date de contact. Nu interacționează cu logica aplicației și nu utilizează context sau state intern.
- **PlaceOrder.jsx**
Componenta PlaceOrder este responsabilă pentru colectarea informațiilor necesare plasării unei comenzi: adresă de livrare, detalii de contact și eventuale observații. Preia datele din StoreContext și le trimite către backend printr-un request POST. Este una dintre componentele mai funcționale, dar poate fi descrisă sumar, deoarece logica sa este derivată

direct din coșul de cumpărături și autentificare.

- Order.jsx

Aceasta componentă apare după ce o comandă a fost finalizată cu succes. Afișează un mesaj de confirmare și, opțional, un sumar al comenzii sau un cod de urmărire. Este simplă din punct de vedere logic, având rol pur informativ.

4.2.8 Admin Panel

Pentru a gestiona eficient produsele și comenzile din aplicația de food delivery, a fost implementată o interfață dedicată administratorului, numită generic Admin Panel. Aceasta este o aplicație React separată de interfața utilizatorului final și are rolul de a facilita operațiuni precum adăugarea de produse, vizualizarea listelor de preparate sau urmărirea comenzilor.

Structura proiectului poate fi observată în figura de mai jos:

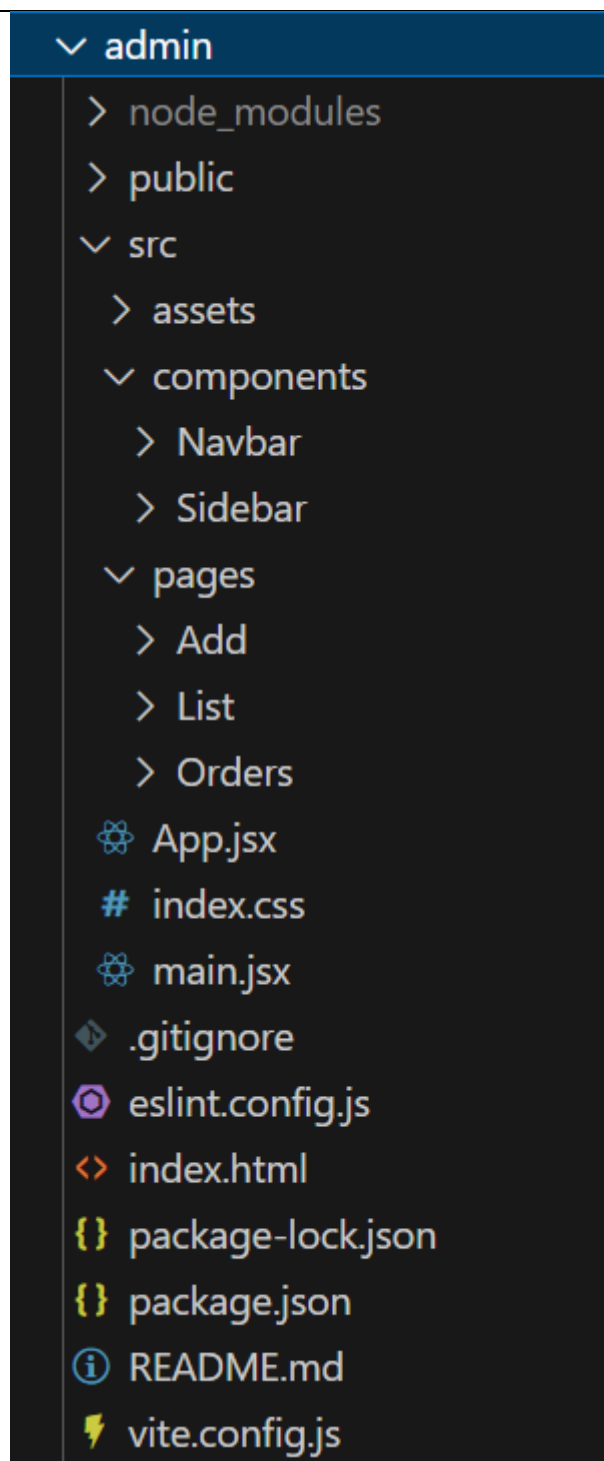


Figura 11 Structura Admin

- src/components

Conține componente reutilizabile:

Navbar – bara de navigare din partea superioară.

Sidebar – meniul lateral pentru navigare între secțiuni.

- src/pages

Conține paginile principale:

Add – formularul pentru adăugarea unui nou produs.

List – afișează toate produsele existente.

Orders – afișează comenzile înregistrate de clienți.

- App.jsx – componenta principală care gestionează routing-ul între pagini.
- main.jsx – punctul de intrare al aplicației.

Componentele de navigație sunt reprezentate de:

- Navbar
Afișează titlul paginii curente și eventuale informații de stare sau notificări.
- Sidebar
Oferă acces rapid la funcționalitățile principale: Add Product, Product List, Orders.

Navigarea între pagini se face printr-un sistem de routing bazat pe react-router-dom.

Admin Panel-ul oferă următoarele funcționalități:

- Adăugarea unui produs nou
In pagina Add, administratorul completează un formular cu datele produsului (nume, descriere, preț, imagine, categorie). La trimiterea formularului, datele sunt transmise către backend pentru stocare.
- Vizualizarea produselor existente
In pagina List, administratorul poate vizualiza toate produsele din baza de date, fiecare cu opțiuni de ștergere sau editare (dacă este implementată).
- Vizualizarea comenzilor
In pagina Orders, sunt afișate toate comenzile efectuate de utilizatori, împreună cu detalii precum numele clientului, produsele comandate, prețul total și statusul comenzii.

Separarea interfeței de administrare de cea a utilizatorului final este o practică frecvent întâlnită în aplicațiile web moderne. Aceasta permite un control mai bun asupra funcționalităților sensibile și oferă o experiență dedicată celor care gestionează sistemul, fără a afecta experiența utilizatorului obișnuit.

4.3 BACKEND

Backend-ul aplicației a fost dezvoltat în limbajul JavaScript, folosind platforma Node.js și framework-ul Express.js. Rolul principal al acestuia este de a gestiona logica aplicației pe partea de server, comunicarea cu baza de date și procesarea solicitărilor HTTP trimise de frontend (atât cel pentru utilizatori, cât și pentru administratori).

- Principalele responsabilități ale backend-ului sunt:
- Gestiunea utilizatorilor (înregistrare, autentificare)
- Gestionarea produselor (creare, listare, ștergere – pentru administrator)
- Preluarea și salvarea comenzilor
- Interacțiunea cu baza de date MongoDB
- Procesarea plăților prin Stripe
- Protejarea anumitor rute prin sistemul de autentificare bazat pe JWT (JSON Web Token)

Backend-ul este organizat într-o structură modulară, care permite scalabilitate și mentenanță ușoară, fiind format din rute, controllere, modele și fișiere de configurare.

4.3.1 Structura proiectului backend

Structura este gândită clar și logic, cu separarea responsabilităților:

- server.js – punctul de intrare al aplicației backend
- routes/ – definește toate rutele API (user, product, order etc.)
- controllers/ – implementează logica fiecărei rute
- models/ – definește structura datelor stocate în MongoDB prin Mongoose
- middleware/ – conține funcții de protecție a rutelor (ex: authMiddleware)

- config/ - setări pentru baza de date și alte constante
- uploads/ - stocare locală temporară pentru fișiere (imagini de produse)

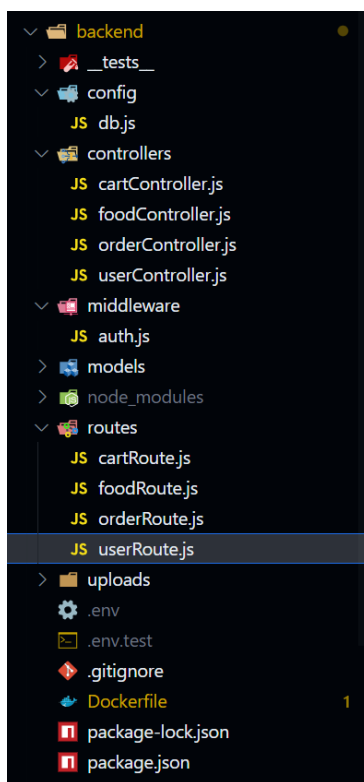


Figura 12 Structura Backend

4.3.2 Descrierea principalelor rute și funcționalități

Aplicația backend este construită modular, iar rutele sunt grupate tematic, fiecare cu rol clar în funcționarea aplicației. Cele mai importante categorii de rute sunt:

Rute pentru utilizator (/api/user) – Aceste rute gestionează înregistrarea și autentificarea utilizatorilor:

- POST /api/user/register – Primește nume, email și parolă. Creează un nou utilizator criptând parola (cu bcrypt) și returnează un token JWT dacă înregistrarea este cu succes.
- POST /api/user/login – Verifică existența utilizatorului, compară parola cu hash-ul stocat, și generează un JWT dacă datele sunt corecte.

Aceste rute sunt esențiale pentru fluxul de login/register din aplicație.

Rute pentru produse (/api/food) – Se ocupă de afișarea și gestionarea produselor alimentare:

- GET /api/food/list – Returnează toate produsele disponibile din baza de date.

- POST /api/food/add – Permite adăugarea unui nou produs (doar de către administrator), inclusiv încărcarea unei imagini.
- POST /api/food/remove – Elimină un produs pe baza ID-ului său.

Uploadul fișierelor se face folosind multer, iar imaginile sunt salvate temporar în folderul uploads/.

Rute pentru comenzi (/api/order) – Aceste rute procesează comenzile trimise de clienți:

- POST /api/order/place – Primește datele comenzii (produse, total, adresa, email) și le salvează în MongoDB.
- POST /api/order/userorders – Returnează comenzile asociate tokenului create în momentul logării utilizatorului.
- GET /api/order/list – Returnează toate comenzile pentru a fi vizualizate de admin în dashboard.
- POST /api/order/status – Returnează statusul comenzii date de către client.
- POST /api/order/verify – Verifică dacă comanda este validă și redirecționează către pagina de comenzi a clientului.

4.3.3 Conectarea la baza de date (MongoDB)

Pentru stocarea datelor (utilizatori, produse, comenzi), aplicația utilizează MongoDB, o bază de date NoSQL flexibilă, orientată pe documente.

Conectarea se face în fișierul db.js:

```
import mongoose from "mongoose";
import dotenv from 'dotenv';

dotenv.config();
const MONGO_URI = process.env.MONGO_URI;

export const connectDB = async () => {
  await mongoose.connect(MONGO_URI).then(()=>console.log("DB Connected"));
}
```

Figura 13 db.js

Conexiunea este asincronă și protejată prin variabile de mediu, stocate în fișierul .env. La inițializarea serverului, se creează o conexiune activă cu baza de date.

Modelele definite în models/ sunt:

- userModel.js – definește schema utilizatorilor: nume, email, parolă (criptată), etc.
- foodModel.js – definește produsele: nume, descriere, imagine, preț, categorie.
- orderModel.js – definește comenzile: conținutul coșului, totalul, data, emailul clientului etc.

Fiecare model este creat cu ajutorul bibliotecii mongoose, ceea ce permite validarea datelor și conversia automată între obiecte JavaScript și documente MongoDB.

4.3.4 Integrarea cu Stripe pentru plăți

Pentru procesarea plăților online, aplicația utilizează serviciul Stripe, o soluție populară și securizată pentru procesarea cardurilor bancare.

Funcționalitate:

- Când utilizatorul apasă butonul "Proceed to Checkout", frontend-ul trimite cererea către backend.
- Ruta POST /api/order/place primește produsele și totalul comenzii.
- Se creează o sesiune Stripe cu stripe.checkout.sessions.create(...).
- Dacă sesiunea este creată cu succes, utilizatorul este redirecționat către pagina Stripe de plată.

```

const placeOrder = async (req, res) => {
  ,

  let session_params = {
    line_items: lineItems,
    mode: "payment",
    success_url: `${frontendUrl}/verify?success=true&orderId=${newOrder._id}`,
    cancel_url: `${frontendUrl}/verify?success=false&orderId=${newOrder._id}`
  };

  if(req.body.discountAmount > 0) {
    let couponName = req.body.discountCode || 'Promo';

    const coupon = await stripe.coupons.create({
      name: couponName,
      amount_off: Math.round(req.body.discountAmount * 100 * 4.5),
      currency: "ron",
      duration: "once"
    });

    session_params.discounts = [{coupon: coupon.id}];
  }

  if (req.body.discountCode === "ANDISTEFANFREE") {
    const totalAmount = getTotalProductsAmount(req.body.items) * 4.5;
    const coupon = await stripe.coupons.create({
      name: "ANDISTEFANFREE",
      percent_off: 95,
      duration: "once"
    });
    session_params.discounts = [{coupon: coupon.id}];
  }
  const session = await stripe.checkout.sessions.create(session_params);

  res.json({
    success: true,
    session_url: session.url,
  });
} catch (error) {
  console.log(error);
  res.json({
    success: false,
    message: "Error",
  });
}
};

```

Figura 14 Integrare StripeAPI:crearea unei sesiuni de plată pentru checkout

4.3.5 Autentificarea și protejarea rutelor (JWT + middleware)

Pentru a securiza datele și a permite doar accesul utilizatorilor autentificați, aplicația folosește JWT (JSON Web Tokens). La autentificare, backend-ul generează un token care este trimis către client și salvat în localStorage. Acest token este apoi trimis în antetul cererilor pentru accesarea rutelor protejate.

La login, token-ul este generat astfel:

```

const createToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET);
};

```

Figura 15 Generare token JWT la autentificare

Middleware-ul authMiddleware verifică validitatea tokenului:

```
const authMiddleware = async (req, res, next) => {
  const {token} = req.headers;
  if (!token) {
    return res.json({
      success: false,
      message: 'Not authorized. Login again.'
    })
  }
  try {
    const tokenDecode = jwt.verify(token, process.env.JWT_SECRET);
    if(!req.body){
      req.body = {};
    }
    req.body.userId = tokenDecode.id;
    req.user = {id: tokenDecode.id};
    next();
  } catch (error) {
    console.log(error);
    return res.json({
      success: false,
      message: 'Error'
    })
  }
}
```

Figura 16 Middleware-ul protejează rutele de acces neautorizat

Rutele protejate (ex: GET /api/order/userorders) utilizează acest middleware:

```
orderRouter.post("/userorders", authMiddleware, userOrders);
```

Figura 17 Utilizarea middleware-ului pentru ruta „userorders”

4.4 CONCLUZII FINALE ALE IMPLEMENTĂRII

Aplicația realizată integrează toate componentele esențiale ale unei soluții moderne de livrare mancare:

- Frontend-ul oferă o interfață dinamică, intuitivă și responsive, construită în React.js, care permite utilizatorilor să navigheze, să comande și să efectueze plăți cu ușurință.
- Backend-ul, dezvoltat în Node.js și Express, gestionează toate procesele critice: autentificare, administrarea produselor, procesarea comenzilor și integrarea cu Stripe pentru plăți securizate.
- Baza de date MongoDB stochează structurat informații despre utilizatori, produse și comenzi, permițând o scalabilitate eficientă.
- Admin Panel-ul separat permite gestionarea aplicației de către personal autorizat, fără a afecta experiența utilizatorului final.

Pe parcursul implementării, proiectul a fost structurat clar, modular, respectând principiile arhitecturii MVC și separării logicii aplicației. De asemenea, s-au utilizat bune practici în organizarea codului, reutilizarea componentelor și securizarea accesului prin JWT și middleware.

5 EXPERIENȚA UTILIZATORULUI

5.1 PAGINA PRINCIPALĂ ȘI NAVIGAREA

După accesarea aplicației, utilizatorul este întâmpinat de o interfață intuitivă și aerisită. Pagina principală este organizată în jurul unui sistem de navigație tipic aplicațiilor moderne: o bară superioară de meniu (Navbar) și o secțiune centrală unde sunt afișate produsele disponibile.

În partea de sus se regăsesc următoarele elemente:

- Logo-ul aplicației;
- Link-uri către secțiunile Home, Menu, Contact;
- Butonul pentru autentificare (Login);
- Icon-ul coșului de cumpărături cu indicator vizual pentru numărul de produse.

Produsele sunt afișate direct pe prima pagină, grupate în funcție de categorie. Utilizatorul poate începe imediat interacțiunea, fără a fi necesară autentificarea pentru a naviga prin meniu.

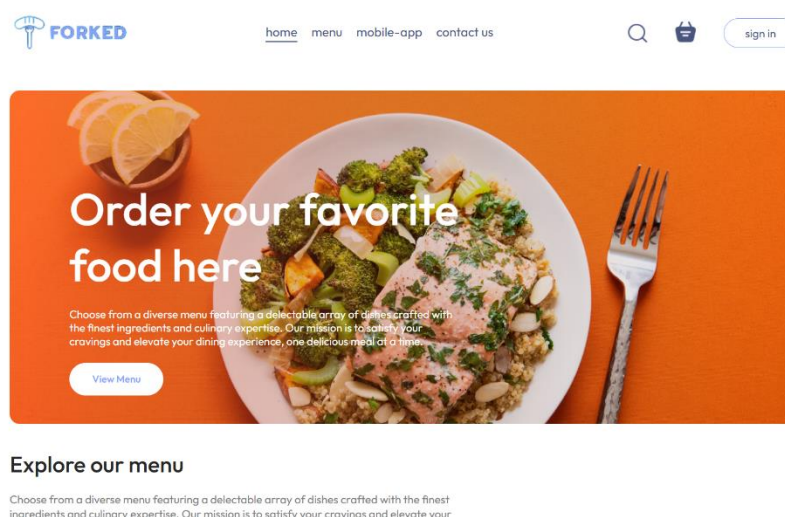


Figura 18 Pagina principală a aplicației

5.2 AUTENTIFICARE ȘI CREARE CONT

Autentificarea este un pas important pentru accesarea funcționalităților care implică personalizarea experienței (comenzi, istoric etc.).

Apăsând pe butonul Login, utilizatorului i se afișează o fereastră de tip pop-up – componenta LoginPopup.jsx. Aceasta permite atât autentificarea cu un cont existent, cât și crearea unui cont nou.

Formularul include următoarele câmpuri:

- Adresă de e-mail;
- Parolă;
- Nume, în cazul înregistrării.

După completarea formularului, datele sunt trimise către backend, care returnează un token JWT în caz de succes. Tokenul este salvat local (în localStorage), iar utilizatorul este considerat „autentificat”. Interfața se actualizează automat – butonul Login este înlocuit cu Logout, iar utilizatorul poate continua să comande.

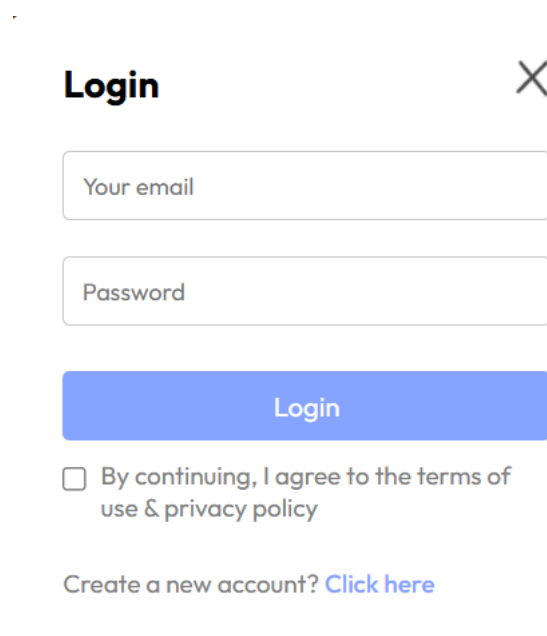
The image shows a 'Login' popup window with a close button (X) in the top right corner. It contains two input fields: 'Your email' and 'Password'. Below these is a blue 'Login' button. Under the button is a checkbox with the text 'By continuing, I agree to the terms of use & privacy policy'. At the bottom, there is a link that says 'Create a new account? Click here'.

Figura 18 Fereastra de autentificare

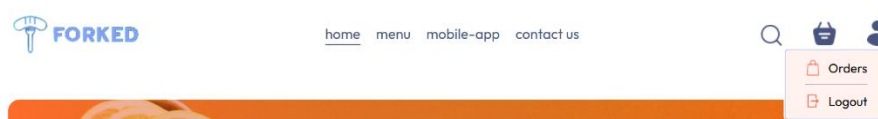


Figura 19 Interfața Navbar, după autentificare

5.3 NAVIGAREA PRIN MENU ȘI FILTRAREA PRODUSELOR

După autentificare sau direct din pagina principală, utilizatorul poate explora meniul aplicației, care conține o gamă variată de produse (ex: pizza, burgeri, băuturi etc.). În partea superioară a secțiunii de meniu, este disponibil un panou de filtrare ce afișează categoriile existente.

Componentele ExploreMenu.jsx și FoodDisplay.jsx colaborează pentru a afișa doar produsele corespunzătoare categoriei selectate. Acest lucru permite utilizatorului să navigheze eficient prin listă, fără a fi copleșit de opțiuni irelevante.

La schimbarea categoriei, lista de produse se actualizează automat, oferind o experiență fluidă și rapidă.

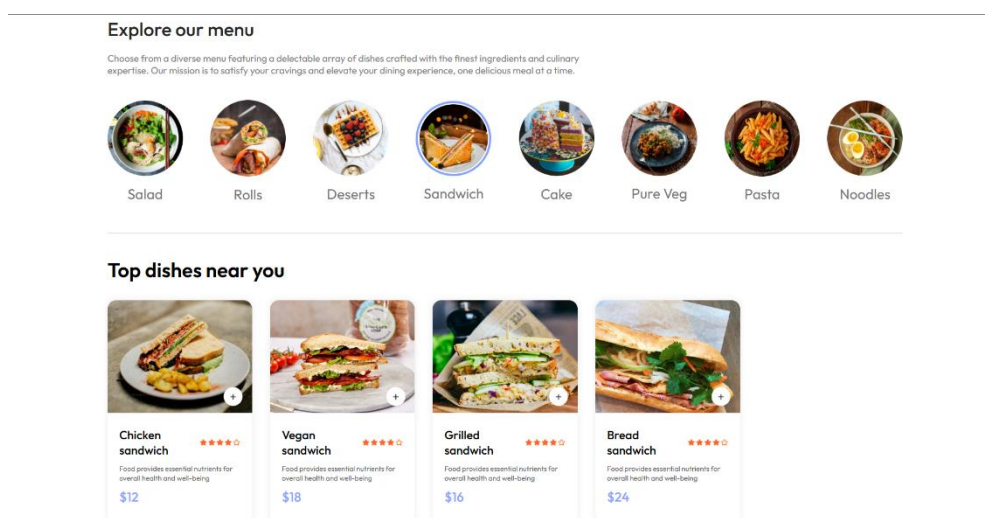


Figura 19 Navigarea prin meniu și afișarea produselor dintr-o categorie selectată

5.4 ADĂUGAREA ÎN COȘ ȘI MODIFICAREA CANTITĂȚII

Pe fiecare card de produs afișat, utilizatorul poate observa:

- Imaginea produsului;
- Numele și descrierea;
- Prețul afișat clar;
- Un buton de adăugare în coș (+), care apare la hover sau direct în colțul imaginii.

Odată ce un produs este adăugat în coș, butonul se transformă într-un mic panou de control format din două butoane (+ și -) și o valoare numerică ce indică numărul de produse

adăugate. Această logică este implementată în componenta `FoodItem.jsx` folosind contextul global (`StoreContext`).

Interfața este dinamică: orice modificare de cantitate este reflectată imediat și vizual, fără a fi necesară reîncărcarea paginii.

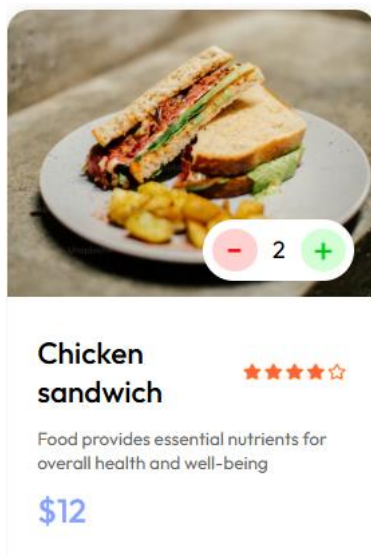


Figura 20 Adăugarea unui produs în coș și modificarea cantității

5.5 VIZUALIZAREA COȘULUI DE CUMPĂRĂTURI

Prin click pe icon-ul coșului de cumpărături din bara de navigare, utilizatorul este direcționat către pagina dedicată (`Cart.jsx`). Aici sunt afișate toate produsele adăugate anterior, împreună cu detalii esențiale:

- Imaginea fiecărui produs;
- Numele și prețul unitar;
- Cantitatea selectată;
- Prețul total per produs;
- Un buton `x` pentru eliminarea produsului din coș.

În partea din dreapta a ecranului este afișat un rezumat al comenzii:

- Subtotal – suma totală a produselor;
- Taxă de livrare – o valoare fixă adăugată dacă există produse în coș;
- Total general – suma finală care urmează a fi achitată.

Toate aceste valori sunt calculate dinamic, folosind funcții definite în context (`getTotalCartAmount()`), iar interfața se actualizează automat la orice modificare a cantității

sau eliminare de produs.

Items	Title	Price	Quantity	Total	Remove
	Greek Salad	\$12	1	\$12	x

Cart Totals		If you have a promo code, enter it here	
Subtotal	\$12	promo code	Submit
Delivery Fee	\$2		
Total	\$14		

Figura 21 Vizualizarea coșului de cumpărături

5.6 PROCESAREA PLĂȚII

După verificarea coșului, utilizatorul poate continua plasarea comenzii prin apăsare butonului „Proceed to Checkout”. Acesta inițiază un request către backend-ul aplicației, care creează o sesiune Stripe.

Stripe redirecționează utilizatorul către o pagină securizată unde poate introduce datele cardului și efectua plata.

Această etapă este complet separată de aplicația principală, pentru a asigura un standard înalt de securitate și conformitate (ex: PCI DSS). După finalizarea tranzacției, utilizatorul este automat redirecționat înapoi către aplicație, pe o pagină de confirmare a comenzii.

←

TEST MODE

Pay

RON 117.00

Chicken sandwich

Qty 2

RON 108.00

RON 54.00 each

Delivery Charges

Qty 1

RON 9.00

Pay with 

Or

Email

email@example.com

Payment method

☐  Card

☐  Klarna

☐ Securely save my information for 1-click checkout

Pay faster on this site and everywhere Link is accepted.

Pay

Powered by 

[Terms](#) [Privacy](#)

Figura 22 Interfața Stripe pentru procesarea plății

5.7 CONFIRMAREA COMENZII

După finalizarea plății, aplicația afișează lista comenzilor date (MyOrders.jsx).

Această pagină semnalează utilizatorului că procesul s-a încheiat cu succes, oferind totodata un feedback pozitiv asupra finalizării unei acțiuni importante.



[home](#) [menu](#) [mobile-app](#) [contact us](#)

My Orders

	Greek Salad x 5, Ripple ice cream x 1	\$76	Items: 2	• Delivered	Track Order
	Chicken salad x 2	\$50	Items: 1	• Out for delivery	Track Order
	Greek Salad x 3	\$38	Items: 1	• Food processing	Track Order
	Cup cake x 2	\$2	Items: 1	• Food processing	Track Order
	Veg salad x 2	\$2	Items: 1	• Food processing	Track Order

Figura 23 Pagina de „MyOrders”

6 CONCLUZII

Realizarea acestei aplicații web de tip food delivery a reprezentat o oportunitate excelentă de a pune în practică cunoștințele acumulate pe parcursul anilor de studiu, dar și de a înțelege în profunzime cum se construiește o aplicație full-stack, cu toate componentele sale esențiale.

Pe parcursul proiectului am acoperit toate etapele majore ale dezvoltării unui sistem complet:

- Frontend-ul, construit cu React.js, a fost gândit pentru a oferi o experiență fluidă, modernă și intuitivă utilizatorilor.
- Backend-ul, dezvoltat în Node.js cu Express, a asigurat logica aplicației, conexiunea cu baza de date și securizarea accesului.
- Baza de date MongoDB a oferit flexibilitate și viteză în manipularea datelor.
- Integrarea cu Stripe a permis procesarea plăților într-un mod securizat și profesionist.
- În plus, a fost realizat și un Admin Panel separat, destinat administratorilor aplicației, pentru gestionarea produselor și comenzilor.

Proiectul nu a fost doar un exercițiu tehnic, ci și o experiență completă de învățare, în care am învățat să îmbin concepte precum arhitectura 3-tier, modelul MVC, comunicarea între componente, validarea datelor, autentificarea prin JWT, gestionarea stării aplicației și protecția rutelor API.

Desigur, aplicația rămâne deschisă pentru îmbunătățiri ulterioare, cum ar fi integrarea cu servicii de livrare, sistemul de recenzii, notificări live sau funcționalități AI pentru personalizarea meniului. Cu toate acestea, scopul lucrării – dezvoltarea unei aplicații funcționale și coerente pentru comenzi online – a fost atins cu succes.

Această lucrare reflectă nu doar cunoștințele teoretice și tehnice acumulate, ci și maturitatea dobândită în planificarea, structurarea și finalizarea unui proiect real, de la zero.

1. ReactJS. React – A JavaScript library for building user interfaces,
<https://reactjs.org/docs/getting-started.html>
2. Express.js. Express – Web framework for Node.js, <https://expressjs.com>
3. MongoDB Inc. MongoDB Documentation. <https://www.mongodb.com/docs>
4. Stripe Inc. Stripe Docs – Payments Integration Guide, <https://stripe.com/docs>
5. Mozilla Developer Network (MDN). JavaScript, HTML, CSS Documentation,
<https://developer.mozilla.org>
6. DigitalOcean. How To Build Modern Web Applications with MERN Stack,
<https://www.digitalocean.com/community/tutorials>
7. FreeCodeCamp. Full Stack Development Articles and Tutorials,
<https://www.freecodecamp.org/news>
8. GeeksforGeeks. Node.js, React, and MongoDB Tutorials,
<https://www.geeksforgeeks.org>
9. LogRocket Blog. Best Practices in React and Node.js Development,
<https://blog.logrocket.com>
10. GitHub – Cod sursă și documentație proiecte de referință,
<https://github.com/AndiTaTaEe/food-delivery-web>
11. W3Schools, <https://www.w3schools.com>